Share Copilot Capacity, Not Accounts

Spec-driven delegation for teams with uneven AI budgets

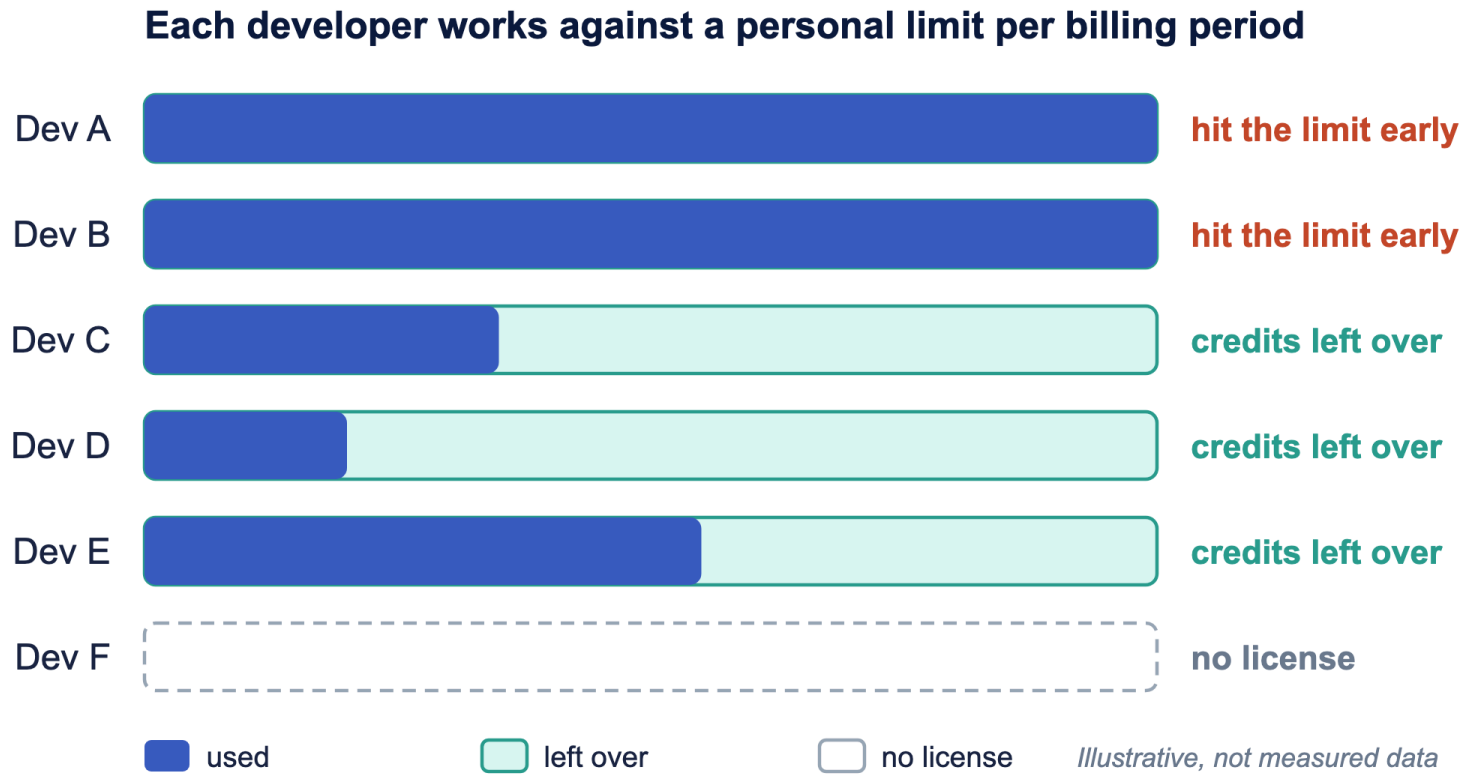
Wallace Espindola
Solution Architect

wallace.espindola@gmail.com
linkedin.com/in/wallaceespindola | github.com/wallaceespindola

The problem: one contract, uneven use

- Copilot usually comes through a **company contract** with Microsoft/GitHub
- Each developer works against a **personal limit** per billing period: a per-user allowance, or a per-user budget admins set on the pool
- Some developers **hit their limit early**, others **end the period with credits left**
- A limit belongs to **one user**: leftover credits do nothing for a colleague who ran out
- Some developers have **no license at all**

Credits the company already paid for



Routing work to people who still have credits gets more value from **the same contract**, without buying extra seats.

"Can you run this through Copilot for me?"

The informal version does not scale:

- The request lives in a **chat message**
- The context is **incomplete**, so Copilot guesses
- The delegate **burns credits** on clarifying questions
- The knowledge **disappears** with the Copilot conversation
- Nobody can see who is waiting on whom

Key principle

Nobody transfers tokens or shares accounts

The owner delegates a **task** to a colleague with Copilot capacity and stays responsible for validating and delivering it.

The **task file**, not the chat history, carries everything the delegate needs.

Why this split works

Costs no AI credits	Where Copilot saves the most time
Understanding the requirement	Writing the implementation
Writing a clear spec	Generating the test suite
Reviewing the spec	Boilerplate, mappers, migrations
Validating the result	Mechanical refactors across files

Owner: thinking and checking. **Delegate:** spends credits only where they pay off.

Spec-driven development, team edition

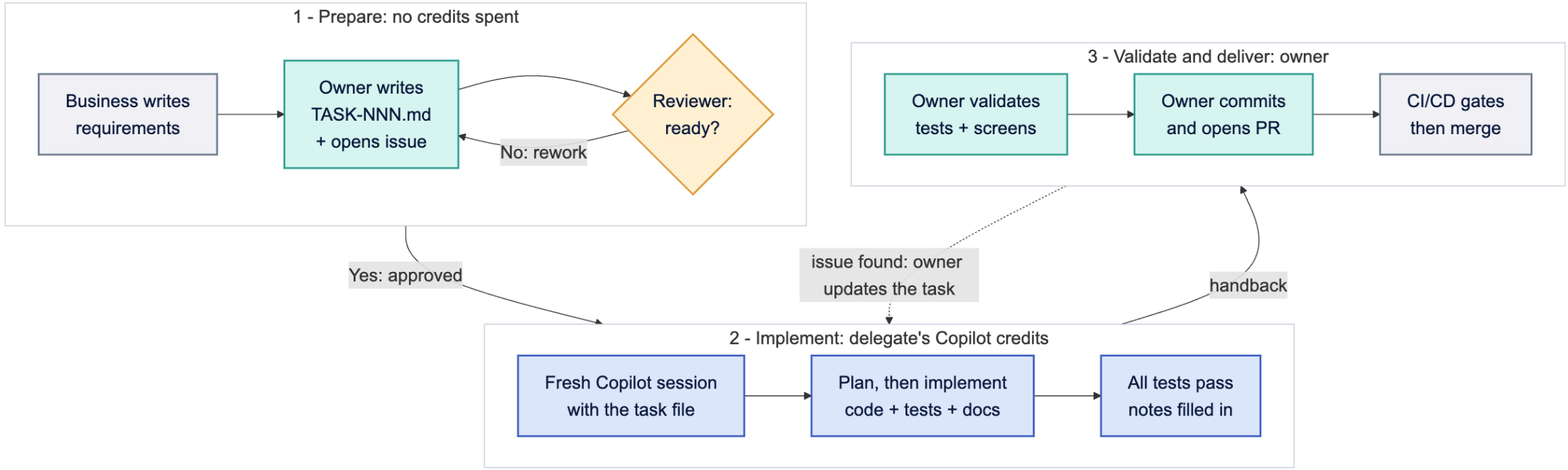
GitHub Spec Kit step	In this process
`/speckit-constitution` (once)	Team standards in `/docs`: architecture, coding, testing guides
`/speckit-specify`	Business requirements, written up by the **owner**
`/speckit-plan` + `/speckit-tasks`	The **approved** `TASK-NNN.md` file
`/speckit-implement`	The **delegate's** Copilot session
`/speckit-converge`	The **owner's** validation loop

Copilot setup: `uv tool install specify-cli` then `specify init <project> --integration copilot`

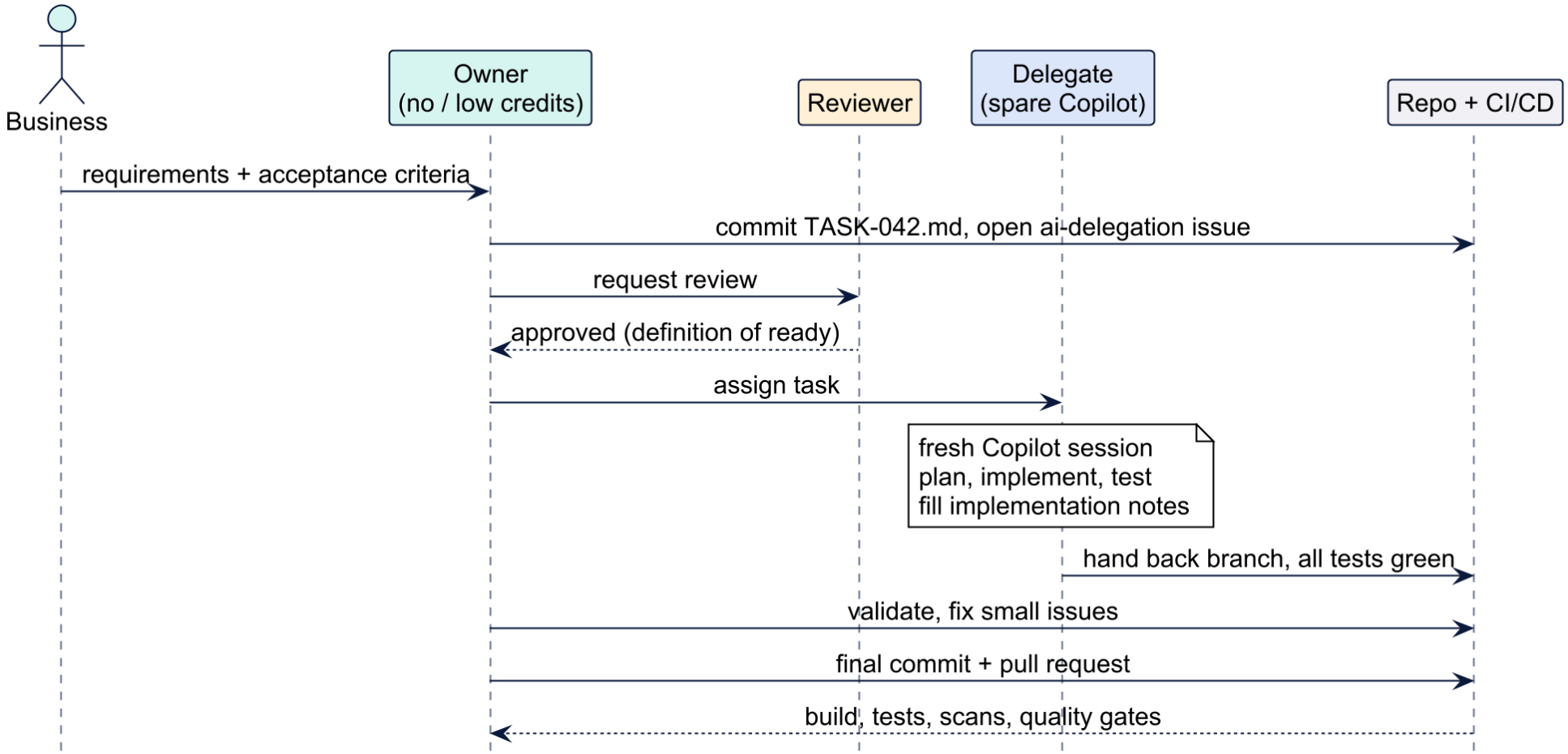
Four roles

Role	Who	Responsible for
Business	Product owner or analyst	Requirements and acceptance criteria
Owner	Dev with no license or a used-up limit	Writing the task, validating, final commit and PR
Reviewer	Architect, tech lead or another dev	Approving the task **before** any credits are spent
Delegate	Dev with Copilot and credits to spare	Implementing with Copilot, including tests and docs

The flow


1 - Prepare: no credits spent
Business writes requirements
Owner writes TASK-NNN.md + opens issue
Reviewer: ready?
No: rework
3 - Validate and deliver: owner
Owner validates tests + screens
Owner commits and opens PR
CI/CD gates then merge
Yes: approved
issue found: owner updates the task
handback
2 - Implement: delegate's Copilot credits
Fresh Copilot session with the task file
Plan, then implement code + tests + docs
All tests pass notes filled in

Who talks to whom



Anatomy of a task file

```
# TASK-042: Generate order summary report
Status: Approved   Owner: <name>   Reviewer: <name>   Delegate: <name>

## Objective
Generate the order summary report from normalized order records.

## Expected behavior
1. Load the report configuration.
2. Retrieve the eligible orders.
3. Generate the report.
4. Store the delivery metadata.

## Acceptance criteria
- [ ] The report is generated for all eligible orders.
- [ ] The job can be restarted after a failure.
- [ ] A failed run is recorded.

## Out of scope
- File transport, customer notification

## Implementation notes   (filled in by the delegate at handback)
```

Definition of ready

The reviewer approves only when **all** of these hold:

- Objective fits in one or two sentences
- Each expected behavior is a **concrete step**, not a wish
- Each acceptance criterion is **checkable** by a test or a visible result
- **Out of scope** is listed
- Every referenced doc **exists and is current**
- Input/output **examples** where the format matters
- **No secrets**, credentials or production data
- Small enough for **one session** (a PR one person reviews in under an hour)

What to delegate, what to keep

Delegate	Keep with the owner
New endpoints, services or jobs with a clear spec	One-line fixes and config changes
Test suites for existing code	Exploratory work, design still open
Mechanical refactors and migrations	Production incidents that need someone now
Boilerplate: DTOs, mappers, repositories	Changes needing access the delegate lacks
Docs generated from existing code	

Delegation has a fixed overhead. It pays off when the work is **large and well defined**.

Naming and tracking

- One file per task: `docs/backlog/TASK-NNN-short-slug.md`
- `NNN` is zero-padded and **never reused**
- Heading repeats ID and title: `# TASK-042: Generate order summary report`
- Each task gets a GitHub issue labeled `ai-delegation` linking the file
- The issue board shows what waits for a delegate and who took what
- Delegates post their capacity: "*can take 2 tasks this sprint*"

Spend credits well

- **One task per session**, started fresh: long sessions carry old context
- Point Copilot at files (`#file`, `#codebase`) instead of pasting them
- Use a **shared** `implement-task` **prompt** so nobody re-explains conventions
- **Plan before editing**: a wrong plan costs one request, a wrong implementation many
- Included models for routine steps, **more expensive models only for the hard parts**
- Copilot stuck in a loop? **Stop and fix the task file**

Repository setup

Asset	Location
Custom instructions	`.github/copilot-instructions.md`
Path instructions	`.github/instructions/*.instructions.md`
Guidance for other agents	`AGENTS.md`
`implement-task` prompt	`.github/prompts/implement-task.prompt.md`
Custom agents	`.github/agents/*.agent.md`
Skills	`.github/skills/<name>/SKILL.md`

Every delegate gets the **same setup** from the repo.

Public agents, skills and MCP servers

Workflow and agents

- **GitHub Spec Kit**: `/speckit-constitution`, `/speckit-specify`, `/speckit-plan`, `/speckit-tasks`, `/speckit-implement`, `/speckit-converge`
- **Copilot agent mode** in the IDE, **Copilot coding agent** on the issue, **Copilot code review** on the PR
- **awesome-copilot**, **anthropics/skills**, **addyosmani/agent-skills**

MCP servers

- **GitHub MCP**: issue, linked PRs, CI results
- **Context7**: current library docs, no outdated APIs
- **Playwright MCP**: drives the UI during validation

Guardrails

- Delegates **already have repository access**: nobody gains access they did not have
- Task files hold **synthetic examples**, never secrets or production data
- Copilot runs only within the company's **approved AI policy** (public code filtering, content exclusions)
- Delegated work passes the **same review, CI/CD and quality gates** as any other change
- `/docs` is the source of truth, **not** Copilot conversations

Is it working? Track per sprint

Metric	What it tells you
Tasks delegated and merged	Is the process being used?
Iterations per task	More than two usually means the task was not ready
Approval to merge time	Compare with similar tasks done by hand
Tasks waiting for a delegate	Growing queue: more capacity or smaller tasks

Key takeaways

- Share **work**, not accounts or tokens
- **Spec-driven development** with the steps split across people
- The **spec** is the handoff, committed in the repo
- **Owners think and check**, delegates spend credits on implementation
- A reviewer gate **before** credits are spent
- Same PR, CI/CD and quality gates as every other change

Resources

Repo (PRD + sample task + design doc):
github.com/wallaceespindola/share-tokens-using-spec-driven-dev

GitHub Spec Kit: github.com/github/spec-kit

Wallace Espindola, Solution Architect
wallace.espindola@gmail.com
linkedin.com/in/wallaceespindola | github.com/wallaceespindola

Thanks! Questions?